Real-data integration plan (Dune snapshots track)

Memo 15

Bloomsbury Tech – Agentic Payments Hackathon Build

April 25, 2026

Date: 2026-04-25 (evening)

> **Status:** parallel to memo 12 (`12_public_data_collection.md`, no-key live track). This memo covers the **Dune snapshots** track — the 4-point trend chart and combined demo. The cofounder's `src/ingest/public_x402.py` + `src/pipeline_public.py` already deliver D1-equivalent (live x402 traffic via x402.watch + Blockscout), so the loader.py plan below is **superseded** by their work — keep this memo as the snapshot-trend reference, not a duplicate of the live track.

After running `scripts/pull_dune.py`, the repo gains three artifacts that need to land into the existing pipeline. This memo lists the minimal edits, ordered by pitch value.

1 Inputs from Dune pull

File	Source	Schema
`data/raw/base_real.parquet`	live `-strategy live`	canonical 14-col + `is_x402`, `facilitator`
`data/raw/base_snapshots.parquet`	`-strategy snapshots`	canonical 14-col + `is_x402` + **`snapshot`** label
`src/ingest/facilitators.py`	auto-generated	constants `CDP_FACILITATOR_POOL`, helpers

2 Three deliverables for Saturday's pitch

Ordered by demo impact — ship D1 first, D2 if time, D3 if extra time.

D1. Live cluster plot on real Base data (highest priority)

Replaces synthetic UMAP with one built on actual x402 traffic. This is the headline visual ("agents and humans separate cleanly") on real data, not a toy.

D2. Snapshot trend slide (most distinctive)

4-point timeline (Jun 2025 \$\to\$ Apr 2026) showing agent population growing, value distribution shifting, counterparty diversity collapsing. Nobody else at the hackathon will have a 12-month trend chart.

D3. Combined demo (synthetic + real overlay)

Synthetic ground truth in background, real Base agents overlaid as a coloured cluster. Demonstrates "same pipeline runs on Base" — pitch beat #4.

3 Architecture

```
SYNTHETIC PATH (existing)             REAL PATH (new)
   src.ingest.synthetic                  scripts/pull_dune.py
        |                                       |
        v                                       v
   data/synthetic/run1.parquet         data/raw/base_real.parquet
                                       data/raw/base_snapshots.parquet
        \_____________________________________/
                          |
                          v
             src.ingest.loader (NEW)
              load(source) -> canonical df

                        v
            src.features.fingerprint (unchanged)
                        v
            src.models.cluster + score (unchanged)
                        v
            src.viz.embed + dashboard (extended)
                        v
            src.viz.trend (NEW -- for snapshots)
```

The downstream stages stay untouched because the schema is canonical. Adding a new source means writing a reader, not touching `features` / `cluster` / `score` / `embed`.

4 File-by-file changes

NEW: `src/ingest/loader.py`

Unified entry point for any data source. Handles canonical-schema parquet from synthetic OR real ingest.

```
"""Unified loader for synthetic, real, and snapshot data.

All sources produce the same 14-col canonical schema; this module
just routes to the right reader.
"""
from __future__ import annotations
from pathlib import Path
import pandas as pd

CANON_COLS = [
    "block_time", "block_time_s", "block_number", "tx_hash",
    "from_addr", "to_addr", "value_usd", "value_wei",
    "gas_used", "gas_price_gwei", "method_id", "success",
    "is_x402", "facilitator",
]

def load(source: str, root: Path = Path(".")) -> pd.DataFrame:
    """source = 'synthetic' | 'real' | 'snapshots' | 'combined' | <path.parquet>."""
```

```
    if source == "synthetic":
        return _ensure(pd.read_parquet(root / "data/synthetic/run1.parquet"))
    if source == "real":
        return _ensure(pd.read_parquet(root / "data/raw/base_real.parquet"))
    if source == "snapshots":
        return _ensure(pd.read_parquet(root / "data/raw/base_snapshots.parquet"))
    if source == "combined":
        syn = _ensure(pd.read_parquet(root / "data/synthetic/run1.parquet"))
        real = _ensure(pd.read_parquet(root / "data/raw/base_real.parquet"))
        syn["data_source"] = "synthetic"
        real["data_source"] = "real"
        out = pd.concat([syn, real], ignore_index=True)
        epoch = out["block_time"].min()
        out["block_time_s"] = (out["block_time"] - epoch).dt.total_seconds()
        return out.sort_values("block_time_s").reset_index(drop=True)
    return _ensure(pd.read_parquet(source))


def _ensure(df: pd.DataFrame) -> pd.DataFrame:
    """Backfill missing canonical columns with safe defaults."""
    for col, default in [
        ("is_x402", False),
        ("facilitator", None),
        ("block_number", 0),
        ("tx_hash", ""),
    ]:
        if col not in df.columns:
            df[col] = default
    return df
```

EDIT: `src/ingest/synthetic.py` (DONE 2026-04-25)

`is_x402` and `facilitator` already emitted via `_emit()`. Per-policy values:

- `gen_human_random` \$\to\$ `is_x402=False`
- `gen_agent_arb_deterministic` \$\to\$ `is_x402=False` (DEX arb, not payments)
- `gen_agent_payment_bot` \$\to\$ **`is_x402=True`**, `facilitator=rng.choice(["cdp", "payai", "thirdweb"])`
- `gen_agent_compromised` \$\to\$ phase 1 inherits from payment_bot; phase 2 `is_x402=False` (drain via approve)
- `gen_collusion_ring` \$\to\$ `is_x402=False`

EDIT: `src/pipeline.py`

Add `-source` arg. Branch: if synthetic, generate as before; otherwise load via `loader`.

```
def run_pipeline(source: str = "synthetic", out_dir: str = "data", **gen_kwargs) -> dict:
    ...
    if source == "synthetic":
        txs, labels = generate(**gen_kwargs)
        txs.to_parquet(tx_path); labels.to_parquet(label_path)
    else:
        from src.ingest.loader import load
        txs = load(source)
```

```
        labels = pd.DataFrame()  # no ground-truth for real data

    features = compute_features(txs)
    ...rest unchanged...
```

CLI: `python -m src.pipeline -source real` runs the full pipeline against `base_real.parquet`. `run_pipeline_from_parquet()` already exists from the morning — fold it into the unified `-source` branch.

NEW: `src/viz/trend.py`

Per-snapshot aggregations + line/area plots for the trend slide. **D2 pitch artifact.**

```
"""Snapshot trend aggregations for pitch slide.

Reads data/raw/base_snapshots.parquet (with ‘snapshot‘ column) and produces:
  - per-snapshot summary table (unique payers, volume, avg tx, counterparty diversity)
  - 4-panel matplotlib figure: agents-over-time, volume, avg tx, diversity
  - plotly version for dashboard
"""
import pandas as pd
import matplotlib.pyplot as plt
from src.features.fingerprint import compute_features

SNAPSHOT_ORDER = ["early_adopters", "post_linux_fdn", "post_stripe", "current"]
SNAPSHOT_DATES = {
    "early_adopters": "Jun 2025",
    "post_linux_fdn": "Oct 2025",
    "post_stripe":    "Jan 2026",
    "current":        "Apr 2026",
}


def aggregate_snapshots(snapshots_df: pd.DataFrame) -> pd.DataFrame:
    """One row per snapshot with key metrics."""
    rows = []
    for label in SNAPSHOT_ORDER:
        sub = snapshots_df[snapshots_df["snapshot"] == label]
        if len(sub) == 0:
            continue
        feats = compute_features(sub)
        rows.append({
            "snapshot": label,
            "date_label": SNAPSHOT_DATES[label],
            "tx_count": len(sub),
            "unique_payers": sub["from_addr"].nunique(),
            "unique_merchants": sub["to_addr"].nunique(),
            "volume_usd": sub["value_usd"].sum(),
            "avg_tx_usd": sub["value_usd"].mean(),
            "median_tx_usd": sub["value_usd"].median(),
            "counterparty_diversity": feats["counterparty_entropy"].mean(),
            "inter_arrival_cv_median": feats["inter_arrival_cv"].median(),
        })
    return pd.DataFrame(rows)


def trend_figure(agg: pd.DataFrame, out_path: str = "results/figures/trend.png") -> None:
    fig, axes = plt.subplots(2, 2, figsize=(11, 7))
```

```
    axes = axes.flatten()
    metrics = [
        ("unique_payers", "Unique agent wallets", "{:,.0f}"),
        ("avg_tx_usd", "Avg transaction (USD)", "${:.3f}"),
        ("counterparty_diversity", "Counterparty entropy (bits)", "{:.2f}"),
        ("inter_arrival_cv_median", "Inter-arrival CV (median)", "{:.2f}"),
    ]
    for ax, (col, title, fmt) in zip(axes, metrics):
        ax.plot(agg["date_label"], agg[col], "o-", linewidth=2, markersize=9)
        ax.set_title(title)
        ax.grid(alpha=0.3)
        for x, y in zip(agg["date_label"], agg[col]):
            ax.annotate(fmt.format(y), (x, y), textcoords="offset points",
                        xytext=(0, 8), ha="center", fontsize=8)
    fig.suptitle("Agentic payment behaviour, Jun 2025 -> Apr 2026", fontsize=13)
    fig.tight_layout()
    fig.savefig(out_path, dpi=150, bbox_inches="tight")
```

EDIT: `src/viz/dashboard.py` (partial — DONE 2026-04-25)

Sidebar source toggle already lives in `dashboard.main()`. Two more changes for Saturday:

(a) Extend toggle options to `["synthetic", "real", "combined"]` once `base_real.parquet` exists.

(b) Trend tab — when snapshots parquet exists, expose a second tab:

```
tab1, tab2 = st.tabs(["Risk dashboard", "12-month trend"])
with tab2:
    if Path("data/raw/base_snapshots.parquet").exists():
        from src.viz.trend import aggregate_snapshots, trend_figure_plotly
        snaps = pd.read_parquet("data/raw/base_snapshots.parquet")
        agg = aggregate_snapshots(snaps)
        st.dataframe(agg, hide_index=True)
        st.plotly_chart(trend_figure_plotly(agg))
```

NEW: `tests/test_loader.py`

Smoke test — synthetic + real have compatible schemas, combined doesn't crash.

```
def test_canonical_schema_matches():
    from src.ingest.loader import CANON_COLS, load
    syn = load("synthetic")  # assumes pipeline ran once
    assert set(CANON_COLS).issubset(set(syn.columns))

def test_combined_loads():
    if not Path("data/raw/base_real.parquet").exists():
        pytest.skip("real data not pulled yet")
    df = load("combined")
    assert "data_source" in df.columns
    assert df["data_source"].isin(["synthetic", "real"]).all()
```

5 Memo updates

After running on Saturday, append:

- **`memos/16_snapshot_findings.md`** — what the 4-point trend actually shows. Templates:

> Unique agent wallets: X \$\to\$ Y (Nx growth). > Average transaction size: X \$\to\$ Y (compression to micro-payments). > Counterparty diversity: X \$\to\$ Y (concentration on top API providers). > Pitch quote: "agent population grew Nx in 12 months while average payment shrunk Mx — these are not human users."

- **`memos/17_real_data_quality.md`** — early_adopters (Jun 2025) likely has near-zero rows. Honest framing: "x402 didn't scale until late 2025; first usable cohort post-Linux Foundation". Use as part of the trend story, not as a bug.

6 Saturday execution sequence

Time	Step	What you prod
10:00	`pull_dune.py -strategy snapshots -snapshot_row_limit 250`	`base_snapshot`
10:05	`pull_dune.py -strategy live -window_minutes 5 -row_limit_x402 350`	`base_real.par`
10:10	Add `loader.py`, fold `-source` into `pipeline.py`	All 3 sources ru
10:30	`python -m src.pipeline -source real`	Real-data UMA
11:00	Add `trend.py`, generate `results/figures/trend.png`	D2 ready
11:30	Add dashboard tabs	D3 ready
12:00	Sanity check D1+D2+D3 in Streamlit	Demo flows end
12:30	Write findings memo	Pitch numbers
14:00 \$\to\$	Interpretability + pitch rehearsal (per existing plan)	

7 Risks and fallbacks

R1. early_adopters snapshot returns 0 rows. Likely — x402 had near-zero volume in June 2025. Fallback: shift first preset to **2025-08-15** ("post-launch traction") in `SNAPSHOT_PRESETS`. Worst case present 3 snapshots not 4.

R2. Live x402 pull thin (<200 rows in 5min). Quick re-run with `-window_minutes 15 -row_limit_x402 700`. Will eat ~5k datapoints — burns both keys. Acceptable on Saturday.

R3. `compute_features` blows up on real data. Expected vulnerable spots:

- `value_usd_log_std` if all values identical \$\to\$ 0 (fine, just non-discriminative)
- `gas_used_cv` if `gas_used` all 55000 placeholders in snapshots \$\to\$ 0 (fine)
- `coordination_partner_count` is $O(N^2)$; >50k tx will be slow. Mitigation: subsample to 30k random tx for `cluster_wallets`.

R4. Real-data UMAP doesn't separate. This is actually possible — without ground-truth `policy` labels, all real Base wallets look "agent-like" on the same axis. Mitigation: use `combined` source so synthetic humans provide the contrast cluster. Pitch fallback: "real Base traffic clusters tightly because it's all agents — humans don't transact at this rate. The synthetic baseline shows what humans look like by comparison."

R5. Datapoints overrun mid-Saturday. With `-use_cache` everything past the morning pulls is free. If you find yourself needing fresh data and out of budget, copy-paste from Dune UI is the manual fallback (works for \$\leq\$10k rows).

8 What we commit, what we ignore

Principle: raw is gitignored, aggregates and summaries are committed.

Path	Commit?	Reason
`data/raw/dune_results/*.csv`	no	large, regenerable
`data/raw/base_real.parquet`	no	derived
`data/raw/base_snapshots.parquet`	no	derived
`data/raw/dune_results/summary.json`	yes	execution_id passport for free
`data/samples/*.csv`	yes	50-100 rows each, smoke-test s
`src/ingest/facilitators.py`	yes	constants used by code
`scripts/pull_dune.py`, `probe_dates.py`, `analyze_snapshots.py`	yes	reproducibility
`results/figures/trend.png`, `real_umap.png`, `headline.png`	yes	pitch artifacts
`results/tables/snapshot_summary.csv`	yes	small, used by memo 13

9 Post-hackathon notes

This integration leaves three production gaps worth documenting in the deck or follow-up: 1. Real ground-truth labels — Forta API integration for compromise weak labels (memo 10 covers this). 2. Streaming ingest — switch from one-shot Dune pull to a daily cron that appends to a partitioned parquet store. 3. Per-facilitator separation — current `is_x402` is binary; a multi-class label (`cdp_pool`, `bankr`, `payai`, ...) would let the model learn facilitator-specific risk profiles.